

Computational Thinking & Coding — Week 1

What Is Computation? Values, Variables, Strings, Input/Output, and Branching

Course materials adapted from MIT OpenCourseWare, *6.100L Introduction to Computer Science and Programming Using Python* (Fall 2022, Ana Bell), Lectures 1-2 and Recitation 1. Licensed under **CC BY-NC-SA 4.0** · <https://ocw.mit.edu> · This adaptation is shared under the same licence.

0. Start here: one sentence about this course

A computer is a machine that follows step-by-step instructions called an algorithm — and it does exactly what you tell it to do, nothing more.

Think of baking a cake from a box: mix, pour, bake, poke with a toothpick, and repeat until it comes out clean. That is an algorithm — **a sequence of simple steps, a flow of control that decides when each step runs, and a way to know when to stop.**

This week we look at three things:

1. how Python **stores** data (numbers, text, True/False),
 2. how a program **talks to us** (`input` / `print`),
 3. how a program **chooses** between paths (`if` / `else`).
-

1. Week 1 learning objectives

By the end of this week you should be able to:

#	You can...
1	Predict the resulting value and its type for a short arithmetic or text expression, before you run it
2	Tell apart the <i>name</i> of a variable and the <i>value</i> the name currently refers to, and say what an assignment changes and what it leaves unchanged
3	Trace on paper which single branch of a conditional is taken for a given input, and name an input that would send the trace down a different branch
4	Describe in words what happens when you take one character out of a text value, take a run of characters out of it, and join two text values — and state the position numbering used
5	Run a supplied program on your own machine, change one stated part of it, and report what you observed
6	Classify a failure you produced as <i>refuses to start</i> or <i>fails while running</i> , and submit it in the four-item report format

What is deliberately *not* in Week 1: writing a program from a blank screen; loops and repetition (Week 2); floating-point tolerance and bisection (Week 3); memorising syntax spelling (both exams give you a one-page core syntax sheet).

2. Terms to know

Term	One-line definition
algorithm	A sequence of simple steps, a flow of control deciding when each step runs, and a way to know when to stop
object & type	Programs manipulate data objects; each object has a type (<code>int</code> , <code>float</code> , <code>bool</code> , <code>str</code>) that decides what you can do with it
variable	A name bound to an object with <code>=</code> (assignment), e.g. <code>radius = 2.2</code> , so we can reuse the value
expression	Objects combined with operators (e.g. <code>3+2</code>); it evaluates to one value, which has a type
string (<code>str</code>)	A sequence of characters in quotes; you can index (<code>s[0]</code>), slice (<code>s[1:3]</code>), concatenate (<code>+</code>) and repeat (<code>*</code>) it
branching	<code>if</code> / <code>elif</code> / <code>else</code> ; the program runs the first block whose condition is <code>True</code>

3. The three things on your screen

Installation asks you to install several things, and nobody ever says how they relate. Name them separately:

#	Thing	What it is
1	The interpreter	The program that actually <i>runs</i> Python code
2	The editor	Where you <i>type and save</i> your code
3	The saved file	Your code, sitting on disk as a <code>.py</code> file

★ Point at all three on your own screen right now. Ten seconds.

Why this matters: when something goes wrong later, this turns *"my computer is broken"* into *"the editor cannot find the interpreter"*— and the second one is a question somebody can answer.

4. Values, types, and expressions

Every value has a **type**, and the **type** decides what you are allowed to do with it.

```
3          # int      – a whole number
3.0        # float    – a number with a decimal point
True       # bool     – a yes/no value
"3"        # str      – TEXT that happens to look like a number
```

★ `3` and `"3"` are **not the same thing**. This single fact is behind most Week 1 confusion.

An **expression** is values combined with operators. It **evaluates to exactly one value**, and that value has a type of its own.

```
3 + 2      # → 5      (int)
"3" + "2"  # → "32"   (str)  ← joining, not adding
9 > 4      # → True   (bool)
```

Try this before class. Write your prediction down first, then run it: `type(3) · type(3.0) · type("3") · type(9 > 4) · type(3 + 2) · type("3" + "2")`

5. Variables and assignment

```
radius = 2.2
```

Read this as an **action**, not as an equation:

First the right-hand side is worked out. **Then** the name on the left is made to refer to that result.

It has a **direction**. It happens **at a moment in time**. It is not a statement that two things are permanently equal.

The state table — write this down every time

Keep a two-column record while you read code:

name	value now
a	1
b	20
sum_ab	21

When a name is reassigned, **you erase the value cell and write the new one**. ★ **The erasing is the point**. You never erase half of an equation.

We are teaching this notation in Week 1 on purpose. In Week 2 you will use the very same table to trace loops. That way Week 2 introduces **one** new thing (repetition), not two.

6. Strings

A string is a sequence of characters in quotes. It is a **value** before it is anything else.

```
s = "Python"

s[0]      # one character  → "P"
s[1:3]    # a run of chars → "yt"
s + "!"   # joining       → "Python!"
s * 2     # repeating     → "PythonPython"
len(s)    # how many chars → 6
```

Position numbering — draw this on paper

	P	y	t	h	o	n	
	↑	↑	↑	↑	↑	↑	
	0	1	2	3	4	5	6

★ Put the position marks on the boundaries between the boxes, not on the boxes. Then `s[1:3]` is "from mark 1 to mark 3" → the two characters `y` and `t`.

Counting starts at **0**, and a slice **includes the left end and excludes the right end**. Almost every beginner's error here is **exactly one too large**. Draw the strip; do not count in your head.

7. Output and input

```
print("Hello")           # showing a value to a person
name = input("Name? ")   # obtaining a value from a person
```

★★ The single most important sentence in Week 1: Whatever `input()` gives you back is TEXT — a `str` — even when the person typed digits.

```
age = input("Age? ")     # you type 20
age + 5                  # ERROR – you cannot add a number to text
int(age) + 5             # → 25 ← you must convert it first
```

On screen, digits typed by a person look exactly like digits computed by the machine. **Nothing visible marks the type**. That is why this one catches everybody, and why we introduce `input` *after* types.

8. Comparisons and branching

A comparison is an expression whose value is `True` or `False`.

```
pset_time = 22
sleep_time = 8

if (pset_time + sleep_time) > 24:
    print("impossible!")
elif (pset_time + sleep_time) >= 24:
    print("full schedule!")
else:
    leftover = abs(24 - pset_time - sleep_time)
    print(leftover, "h of free time!")

print("end of day")
```

Predict the output before you run it. Then change `pset_time` to `15` and predict again. Is the last line always printed?

Three rules that this example is built to teach

(a) **Exactly one branch runs.** The program does not check every branch and it does not run both. Picture a single coin walking down a fork in the path — it cannot be in two places.

(b) **The first condition that holds wins — so the order matters.** Because exactly one branch is taken, reordering the conditions can change the answer. Try swapping the `if` and the `elif` above and trace it again with the same input.

(c) **Indentation is not decoration — it is the notation that says which steps belong to the branch.**

`print("end of day")` sits at the left margin, so it is *outside* the branch and always runs. If you "tidy up" the alignment, the behaviour changes.

Boundary values

What happens **exactly** at the threshold? Above, try `pset_time = 16, sleep_time = 8`, so the sum is **exactly 24**. Which branch runs? Check `>` against `>=` carefully.

★ Get into the habit now. In Week 2, *missing boundary cases* becomes the single most common failure.

9. Where students usually slip

Common thought	What is actually true
" <code>=</code> means the two sides are equal, like in maths"	It is an action with a direction , performed at a moment in time. Right side first, then the name is bound to the result
"A name remembers the formula it was given, so if an ingredient changes later, the name changes too"	That is a spreadsheet , not a Python program. Once assigned, the value cell is independent
"A variable is the unknown, like the one I solve for"	In maths a variable is an unknown to be solved for . Here a name refers to a value that is already decided — you <i>read</i> it, you do not <i>solve</i> for it
★ "What the program read from the person is a number, because it looks like a number"	It is text . See section 7. This is the most frequent Week 1 collapse
"Position numbering starts at 1 and a slice includes both ends"	It starts at 0 and the right end is excluded . Draw the boundary strip
"The machine checks every branch" / "runs both"	Exactly one branch runs
"The order of the chained conditions does not matter"	The first condition that holds wins. Reordering can change the result
"Indentation is decoration"	It is the notation for block structure. Misindented code is a refuse-to-start failure
★ "An error message means I failed / the machine is broken"	An error message is information , and producing one is how you get help. See section 11
"The editor is the language" / "the program lives on the internet"	Three separate things. See section 3

10. Lab worksheet — R01

Ground rule for the lab: the TA never gives away answers. The TA only asks **tracing questions**.

Warm-up

```
# declaring a simple string
basic_string = _____ # (1) a sentence in double quotes, e.g. with your name

# integers and simple operations
a = 1
b = _____ # (2) assign the integer 20
sum_ab = a _____ b # (3) the sum of the two numbers
```

Then run `type(basic_string)` and `type(a)` in the console and write down what each returns:

_____ / _____

Exercise 1 — integers and simple operations

(a) **Worked example** — read and run this as-is.

```
a = 1
b = 20
sum_ab = a + b          # sum_ab is now 21
difference_ab = b - a    # difference_ab is now 19
```

(b) **One-line change** — change **only** the first line to `a = 5`. Predict before running: `sum_ab = _____`, `difference_ab = _____`

(c) **Fill in the blanks** — complete the code so it computes the same two results.

```
a = 5
b = 20
sum_ab = ____ + ____
difference_ab = ____ - ____
```

TA tracing questions 1. "Right after line 3 runs, what value is stored in `sum_ab`? And in `difference_ab` — does it exist yet?" 2. "Why do `b - a` and `a - b` give different results? What would you change to get `-19`?"

Exercise 2 — `str()` conversion and string concatenation

(a) **Worked example** — run this, then print the variable.

```
# example operations on integers and strings
my_numbers_as_string = str(9) + ' ' + str(10) + ' ' + str(7)
```

(b) **One-line change** — replace `str(7)` with `str(a)` (with `a = 5` from Exercise 1). Predict the printed result before running: _____

(c) **Fill in the blanks** — complete the line so it produces exactly `"9 10 7"`.

```
my_numbers_as_string = ____ (9) + ' ' ____ str(10) + ____ + str(7)
```

TA tracing questions 1. "What is the difference between `9` and `str(9)`? What does `type()` say for each?" 2. "If you remove `str()` and write `9 + ' '`, what happens — and why does Python refuse?"

Self-check

Tick a box only if you can do it **without looking at the completed code**.

- ☐ I can declare a string variable and an integer variable, and explain how their types differ.
- ☐ I can predict the values of `sum_ab` and `difference_ab` for any pair of `a`, `b` before running.
- ☐ I can explain why `str()` is needed before joining a number to a string with `+`.

11. The four-item error report — required from day one

Producing an error is not failure. It is your entry ticket to help.

Whenever you get stuck, bring these four items — this is the required format for asking any question in this course:

#	Item
1	Expected value — what you thought would happen
2	Actual value — what actually happened
3	Minimal reproducing input — the smallest code/input that still shows the problem
4	First line of the traceback — the <i>first</i> line, not the last

And classify the failure as one of two kinds:

- **Refuses to start** — the program never runs at all (e.g. bad indentation, a typo in the syntax)
- **Fails while running** — the program starts and then stops partway (e.g. adding a number to text)

★ Naming the kind is already half of the diagnosis. Do it before you ask.

If you have been stuck for ten minutes and have not written a report card, that is the moment to raise your hand. Silence is the thing we are watching for — not raised hands.

12. Think about this before class

1. The lecture says "*a computer will only do what you tell it to do.*" What everyday task do you follow so precisely that a machine could do it? **Where would it break down?**
2. Writing text in quotes (`"3"`) and a number without quotes (`3`) are different types in Python. Why might it be useful — or dangerous — for a language to treat them differently?

13. Next week

Loops and repetition — and the first real difficulty spike of this course.

- Repetition: asking the program to do something again, and **knowing when to stop**
- The state table you learned this week, applied to a loop
- Boundary cases (the ones we started raising in section 8)

★ **One thing to know in advance:** next week a program can be asked to repeat, and **a program that never stops is a normal, expected outcome** — not a sign that you broke something. When it happens, it is reported like any other failure, with the same four-item report from section 11.

Problem Set 1 (Compound Interest) is assigned in Week 2 — control flow, formulating a computational solution, and bisection search. Part A 30% · Part B 30% · Part C 40%.